Architecture Document

1. Application Architecture Selection: Microservices For the Clinical Decision Support System (CDSS) designed for Cardiac Arrhythmia Detection, a Microservices architecture has been selected. This architectural style was chosen because the system requires distinct, decoupled environments for its various operations. The presentation tier (dashboard) must remain lightweight and responsive for clinicians at the edge, while the machine learning tier (handling the ResNet18 contrastive learning feature extraction and XAI heatmap generation) requires heavy GPU compute resources. By isolating the ML inference engine as an independent microservice, the system can scale compute resources efficiently and update predictive models without causing downtime for the clinical dashboard.

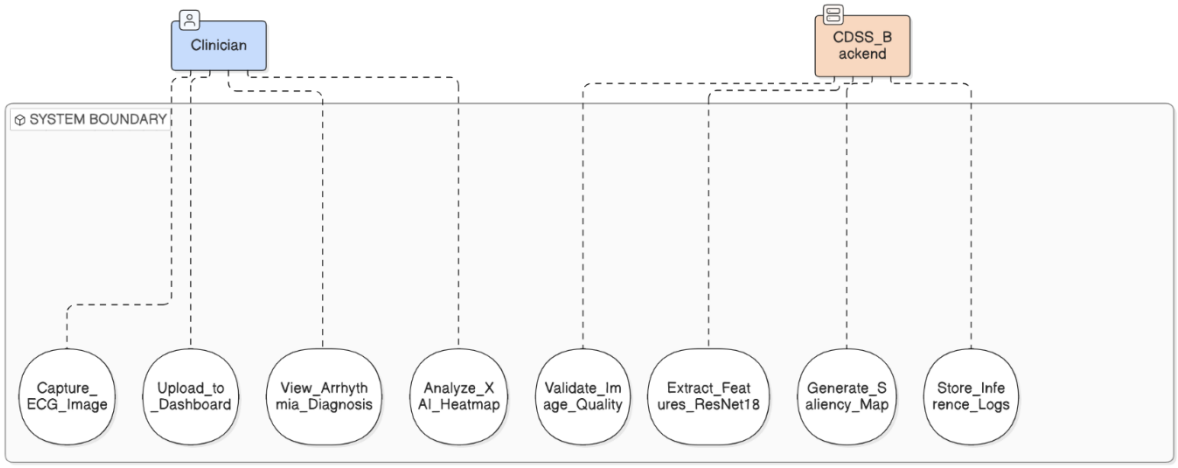


Figure 1: System Use Case Diagram. This diagram defines the system boundaries and actor interactions within the CDSS. The primary actor, the Clinician, is responsible for capturing the physical ECG printouts and utilizing the dashboard to review diagnostic results. The CDSS Backend system autonomously handles the contrastive learning feature extraction, saliency map generation, and rigorous validation processes, ensuring a seamless diagnostic workflow.

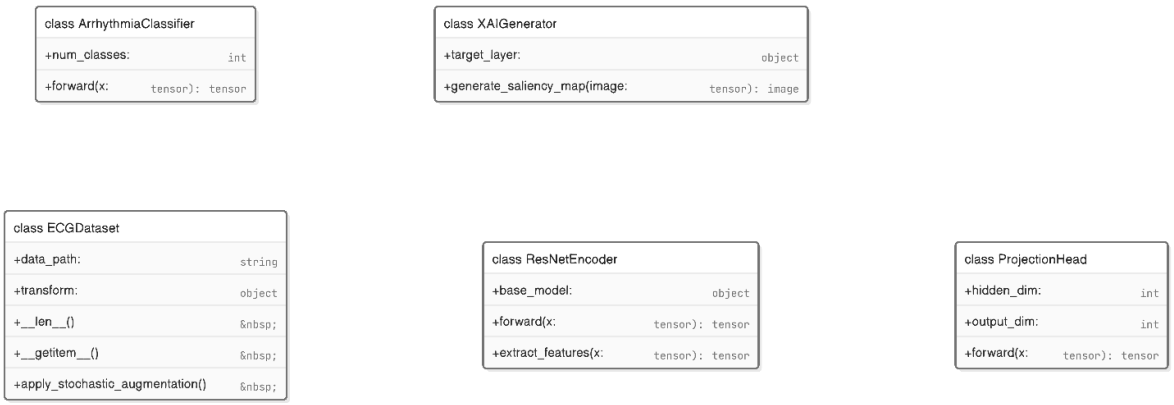


Figure 2: ML Pipeline Class Diagram. This object-oriented class diagram defines the software structure of the deep learning pipeline. It separates the data augmentation processes (ECGDataset) from the core model architecture. The ResNetEncoder serves as the central hub, feeding representations into either the ProjectionHead for Stage 1 pre-training or the ArrhythmiaClassifier for Stage 2 fine-tuning, while also supplying feature gradients to the XAIGenerator.

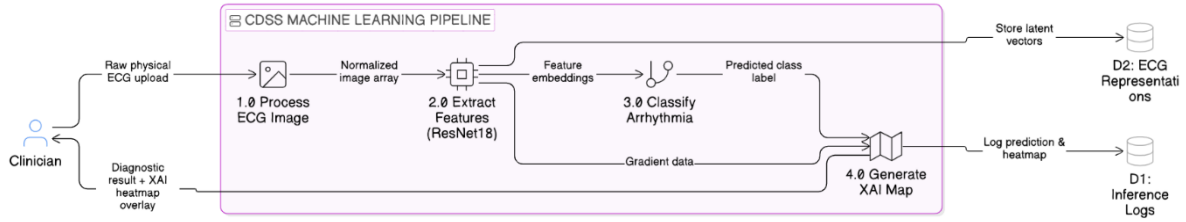


Figure 3: Data Flow Diagram (Level 1). The DFD maps the transformation of the raw ECG image array as it is processed by the system. It tracks the flow of data from the initial edge upload by the clinician, through the ResNet18 vectorization and classification nodes, culminating in the generation of the XAI saliency map and the secure logging of latent vectors and predictions into the database.

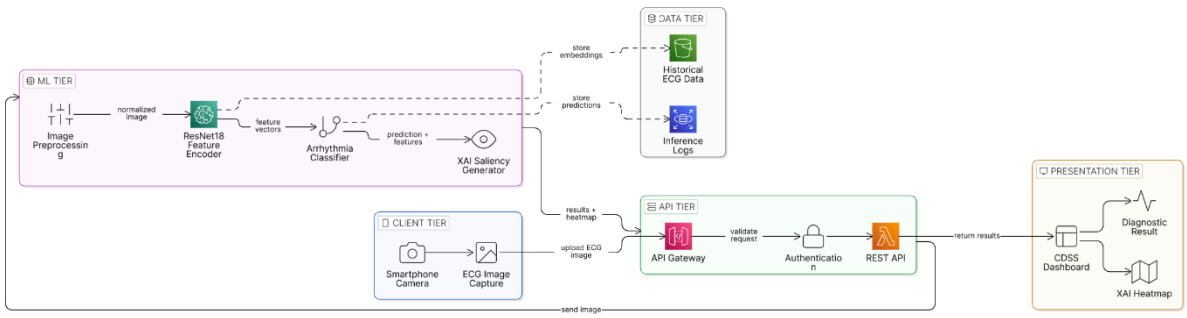


Figure 4: Microservices Deployment Architecture. This diagram illustrates the 5-tier deployment architecture of the CDSS. The Client Tier captures edge data and routes it securely through the API Gateway. The ML Tier acts as an isolated microservice hosting the

fine-tuned encoder and XAI generator. This decoupled design ensures high scalability and allows the deep learning inference engine to be updated independently.

---

2. Database, ER Diagram, and Schema Design The data persistence layer is designed to ensure strict data integrity and compliance with healthcare audit standards.

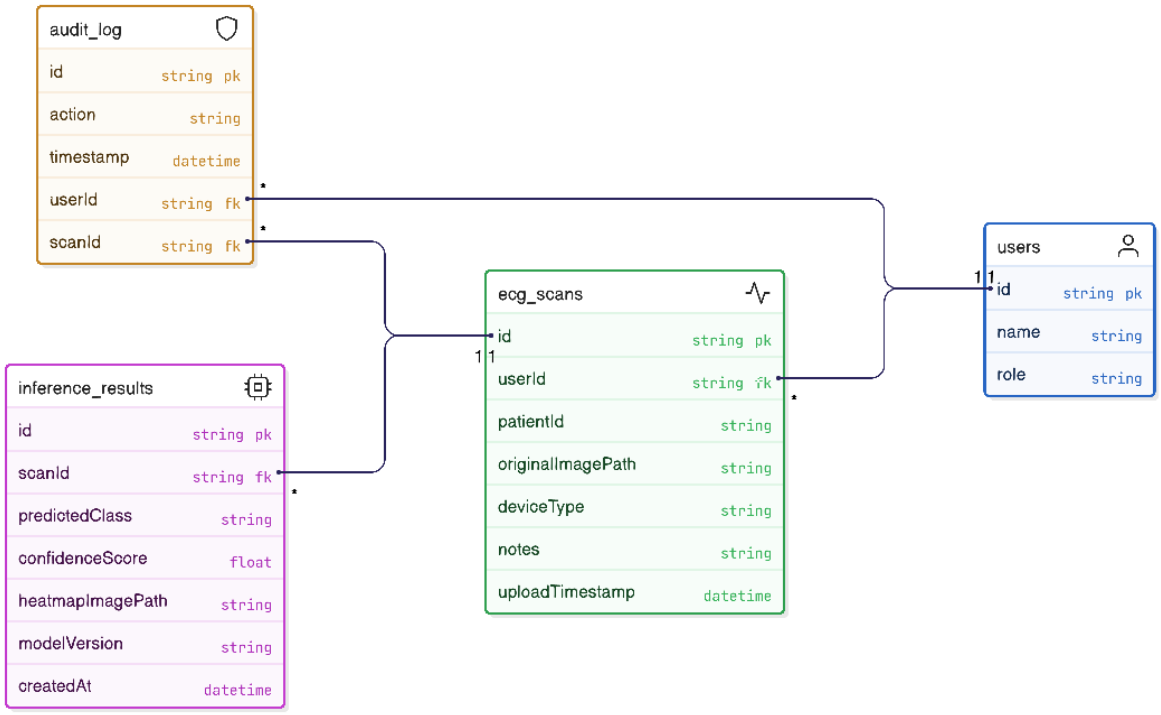


Figure 5: Entity-Relationship Diagram and Schema Design. The database schema utilizes a relational model connecting the users (clinicians and researchers) to their respective ecg_scans. A one-to-many relationship exists between the scans and the inference_results to allow for multiple model versions or clinical second opinions. Crucially, the audit_log table tracks all interactions with sensitive data, guaranteeing systemic transparency and regulatory compliance for medical applications.

---

3. Data Exchange Contract

- **Frequency of data exchanges:** Data exchanges occur synchronously and on-demand whenever a clinician uploads an ECG image for inference.
- **Data Sets:** The payload consists of physical ECG image data sent from the client, and JSON-formatted diagnostic results (predicted class, confidence score, and XAI heatmap image path) returned from the server.
- **Mode of Exchanges:** REST API.

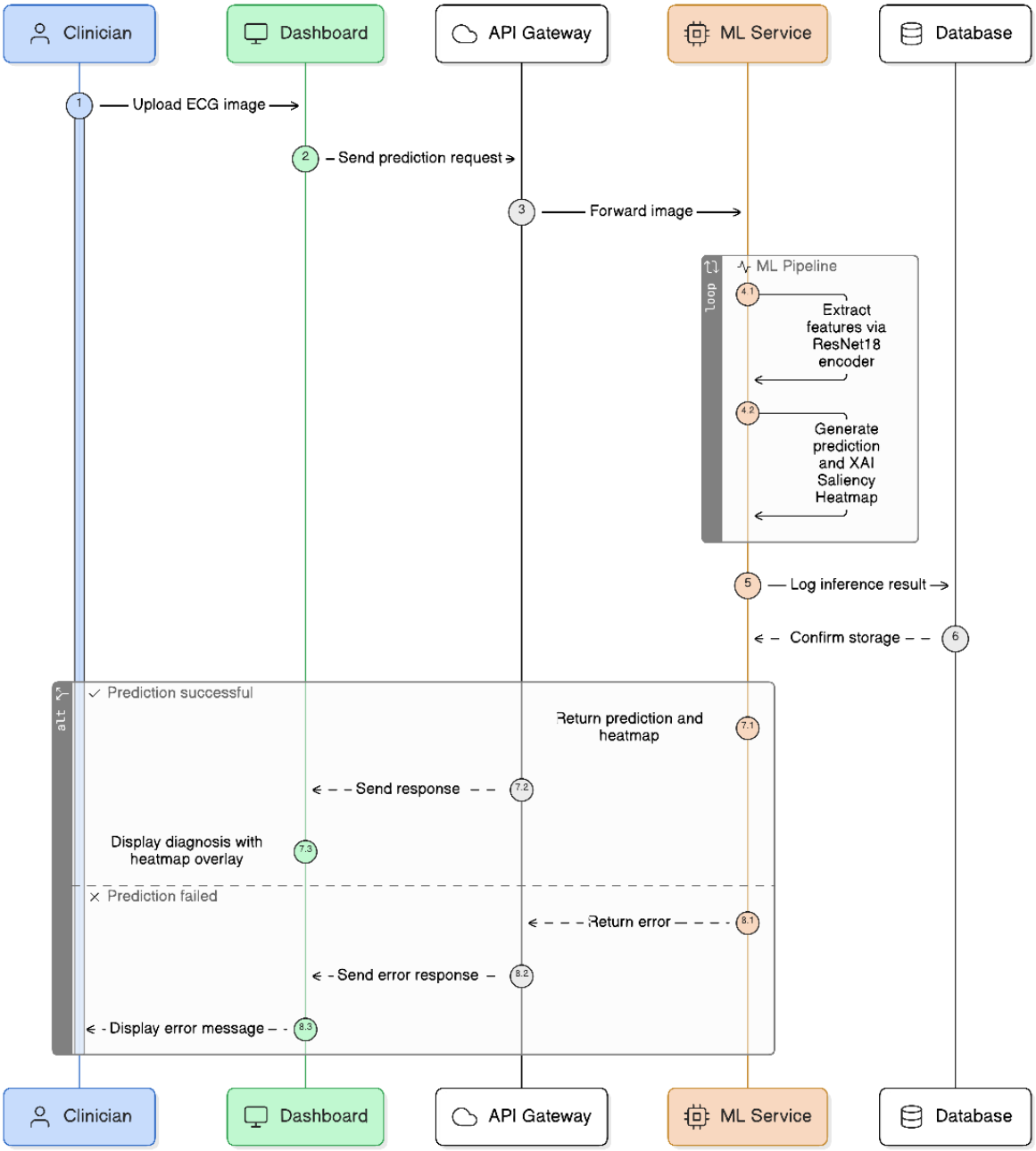


Figure 6: API Sequence and Mode of Exchanges. This sequence diagram maps the mode of data exchange via REST API during a standard clinical inference request. Upon receiving the ECG image payload, the API Gateway triggers the ML pipeline. The architecture handles both successful anomaly detection and failure states, providing an immediate JSON response with the diagnostic result and XAI heatmap overlay back to the clinician's dashboard.